SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE NAME: Cryptography and Network Security

COURSE CODE: CSA5113

Code of Conduct:

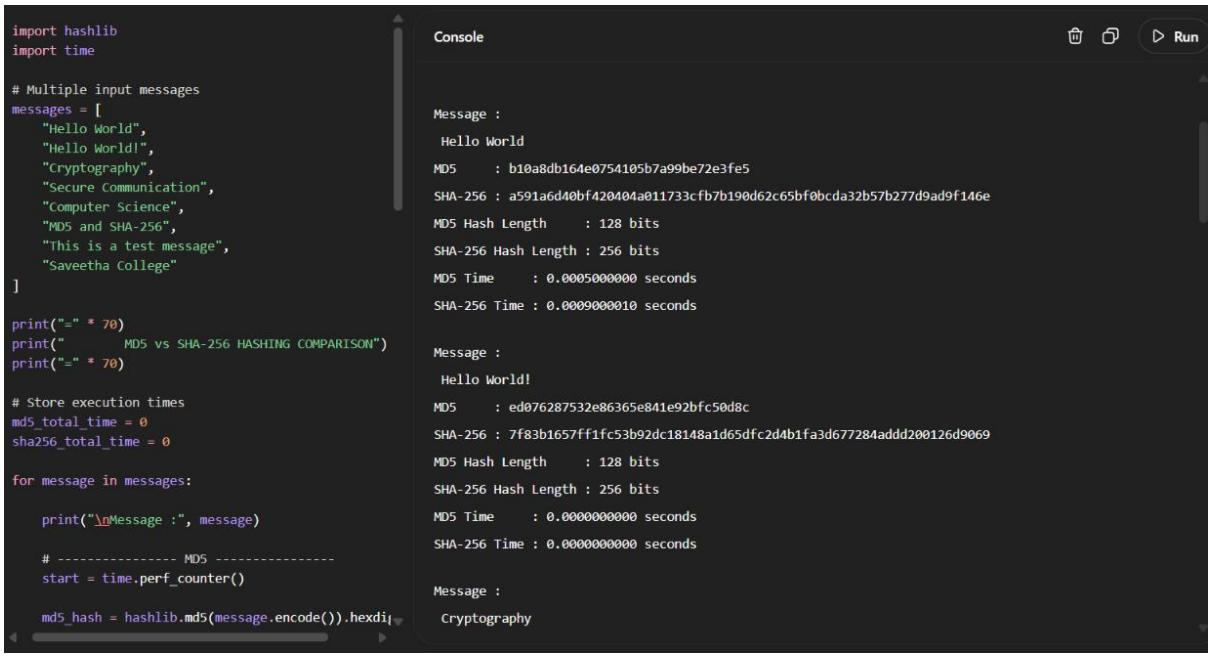
I S,DHIVYASRI(192511246)certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.I understand that any violation of academic integrity rules will result in disciplinary action.

Annexure A

Experiments

1. Implement and compare MD5 and Secure Hash Algorithm (SHA-256) in Python by hashing multiple input messages. Analyze their collision resistance and performance differences

ANSWER:



```
import hashlib
import time

# Multiple input messages
messages = [
    "Hello World",
    "Hello World!",
    "Cryptography",
    "Secure Communication",
    "Computer Science",
    "MD5 and SHA-256",
    "This is a test message",
    "Saveetha College"
]

print("=" * 70)
print("       MD5 vs SHA-256 HASHING COMPARISON")
print("=" * 70)

# Store execution times
md5_total_time = 0
sha256_total_time = 0

for message in messages:

    print("\nMessage :", message)

    # ---------------- MD5 ----------------
    start = time.perf_counter()

    md5_hash = hashlib.md5(message.encode()).hexdi
```

```
Console                                                    Run

Message :
 Hello World
MD5     : b10a8db164e0754105b7a99be72e3fe5
SHA-256 : a591a6d40bf420404a011733cfb7b190d62c65bf0bcda32b57b277d9ad9f146e
MD5 Hash Length     : 128 bits
SHA-256 Hash Length : 256 bits
MD5 Time     : 0.0005000000 seconds
SHA-256 Time : 0.0009000010 seconds

Message :
 Hello World!
MD5     : ed076287532e86365e841e92bfc50d8c
SHA-256 : 7f83b1657ff1fc53b92dc18148a1d65dfc2d4b1fa3d677284addd200126d9069
MD5 Hash Length     : 128 bits
SHA-256 Hash Length : 256 bits
MD5 Time     : 0.0000000000 seconds
SHA-256 Time : 0.0000000000 seconds

Message :
 Cryptography
```

2. Develop a Python program to implement a Digital Signature scheme using RSA.Demonstrate message signing and signature verification, and analyze its effectiveness in ensuring integrity and authentication.

ANSWER:

```python
# RSA Digital Signature Implementation
# Message Signing and Signature Verification

import hashlib
import random
import math


# ---------------------------------------------
# RSA KEY GENERATION
# ---------------------------------------------

def gcd(a, b):
    while b != 0:
        a, b = b, a % b
    return a


def mod_inverse(e, phi):
    return pow(e, -1, phi)


def generate_rsa_keys():
    # Two prime numbers
    p = 61
    q = 53

    # Calculate n
    n = p * q

    # Euler's Totient
```

```
Console                                                    Run

Run started
Initializing environment
Installing packages
Running code
============================================================
          RSA DIGITAL SIGNATURE
============================================================

Public Key  :
 (17, 3233)
Private Key : (2753, 3233)

Original Message:
This is a confidential message.

Digital Signature:
947

Signature Verification:
VALID - Message is authentic and has not been modified.

============================================================
```

```python
def hash_message(message):
    return int(
        hashlib.sha256(message.encode()).hexdiges
        16
    )


# ---------------------------------------------
# SIGN MESSAGE
# ---------------------------------------------

def sign_message(message, private_key):
    d, n = private_key

    # Hash the message
    message_hash = hash_message(message)

    # RSA signature
    signature = pow(message_hash, d, n)

    return signature


# ---------------------------------------------
# VERIFY SIGNATURE
# ---------------------------------------------

def verify_signature(message, signature, public_k
    e, n = public_key

    # Hash received message
    message_hash = hash_message(message)
```

```
Console

Signature Verification:
VALID - Message is authentic and has not been modified.

============================================================
Testing Modified Message
============================================================

Modified Message:
This is a modified message.

Signature Verification:
INVALID - Message has been modified!

============================================================
                    CONCLUSION
============================================================

1. RSA private key is used to create the digital signature.
2. RSA public key is used to verify the signature.
3. SHA-256 is used to generate the message hash.
4. If the message is modified, signature verification fails.
5. Digital signatures provide message integrity and authentication.
Run completed in 2619.9000000953674ms
```

3. Write a Python program to simulate HMAC (Hash-based Message Authentication Code) using a chosen hash function. Compare its security properties with a simple hash-based integrity check.

ANSWER:

```python
import hashlib
import hmac

# ------------------------------------------------
# SIMPLE HASH INTEGRITY CHECK
# ------------------------------------------------

def simple_hash(message):
    return hashlib.sha256(message.encode()).hexdi


# ------------------------------------------------
# HMAC USING SHA-256
# ------------------------------------------------

def generate_hmac(message, key):
    return hmac.new(
        key.encode(),
        message.encode(),
        hashlib.sha256
    ).hexdigest()


# ------------------------------------------------
# MAIN PROGRAM
# ------------------------------------------------

print("=" * 70)
print("       HMAC vs SIMPLE HASH INTEGRITY CHECK
print("=" * 70)
```

```
Console

Run started
Initializing environment
Installing packages
Running code
======================================================================
       HMAC vs SIMPLE HASH INTEGRITY CHECK
======================================================================

Original Message:
Transfer Rs.1000 to Account A

Secret Key:
MySecretKey123

----------------------------------------------------------------------
SIMPLE SHA-256 HASH
----------------------------------------------------------------------
Hash:
dc44143ed4aebef507665f242f4a81876fa170d99d08ca129faeb3334f51ec1e

----------------------------------------------------------------------
```

```python
print("SIMPLE SHA-256 HASH")
print("-" * 70)

print("Hash:")
print(simple_hash_value)


# ------------------------------------------------
# HMAC
# ------------------------------------------------

hmac_value = generate_hmac(message, secret_key)

print("\n" + "-" * 70)
print("HMAC-SHA256")
print("-" * 70)

print("HMAC:")
print(hmac_value)


# ------------------------------------------------
# VERIFY ORIGINAL MESSAGE
# ------------------------------------------------

print("\n" + "=" * 70)
print("VERIFYING ORIGINAL MESSAGE")
print("=" * 70)

# Simple hash verification
received_hash = simple_hash(message)
```

```
Console

SIMPLE HASH:
- Uses only the message as input.
- Anyone can calculate the hash.
- Provides data integrity.
- Does NOT provide authentication.
- An attacker can modify the message and generate a new hash.

HMAC-SHA256:
- Uses a secret key along with the message.
- Provides message integrity.
- Provides message authentication.
- An attacker cannot generate a valid HMAC without the secret key.
- More secure than a simple hash for message authentication.
======================================================================
CONCLUSION
======================================================================

HMAC-SHA256 is more secure than a simple SHA-256 hash.

A simple hash can detect accidental changes, but it cannot prove
that the message came from an authorized sender.

HMAC uses a secret key, so it provides both integrity and
authentication.
Run completed in 1883.2999999523163ms
```

4. Implement a simplified Kerberos authentication mechanism in Python to simulate ticket generation, distribution, and validation between a client and server. Analyze its effectiveness in preventing replay attacks.

ANSWER:

```python
import time
import hmac
import hashlib
import base64

# ================================================
# SIMPLIFIED KERBEROS AUTHENTICATION
# ================================================

# Shared secret keys
CLIENT_KEY = "client_secret_key"
SERVER_KEY = "server_secret_key"

# Ticket validity in seconds
TICKET_LIFETIME = 10


# ================================================
# CREATE HMAC SIGNATURE
# ================================================

def create_signature(data, key):
    return hmac.new(
        key.encode(),
        data.encode(),
        hashlib.sha256
    ).hexdigest()


# ================================================
# AUTHENTICATION SERVER
```

```
Console

Run started
Initializing environment
Installing packages
Running code
======================================================================
              SIMPLIFIED KERBEROS AUTHENTICATION
======================================================================

Client:
 Alice

[Authentication Server]
Authenticating user: Alice
User authenticated successfully.
Session key generated.

[Ticket Granting Server]
Ticket generated successfully.
Ticket:
QWxpY2V8U0VTU0lPTl9BbGljZV8xMjM0NXwxNzg3MTE3NjM3fDA0NmJkZjA4YjU2M2I3ODUwNjBjMDQ3NGRkMzQ3MmRmNmU
3NjY2ODk1NWU1Mzc3OGJkNTBlZmI2YWY1Y2M=
```

```python
# ================================================
# TICKET GRANTING SERVER
# ================================================

def generate_ticket(username, session_key):
    print("\n[Ticket Granting Server]")

    timestamp = int(time.time())

    # Ticket contains client, session key and tim
    ticket_data = (
        username + "|" +
        session_key + "|" +
        str(timestamp)
    )

    # Sign ticket using server secret key
    signature = create_signature(ticket_data, SER

    ticket = ticket_data + "|" + signature

    # Encode ticket
    encoded_ticket = base64.b64encode(
        ticket.encode()
    ).decode()

    print("Ticket generated successfully.")
    print("Ticket:", encoded_ticket)

    return encoded_ticket
```

```
Console                                                    Run

======================================================================
TICKET EXPIRATION TEST
======================================================================

Waiting for ticket expiration...
Trying to use the expired ticket...

[Application Server]
Validating ticket...
Authentication FAILED: Ticket expired.

======================================================================
                        CONCLUSION
======================================================================

1. Kerberos uses tickets instead of sending passwords repeatedly.
2. The ticket contains a session key and timestamp.
3. The server verifies the ticket using a secret key.
4. A previously used ticket is rejected to prevent replay attacks.
5. Ticket expiration limits the lifetime of stolen tickets.
6. Therefore, Kerberos provides authentication and helps prevent
   replay attacks using timestamps and ticket validation.

Run completed in 13043.5ms
```

5. Develop a Python-based implementation of either the ElGamal or Schnorr digital signature scheme, and evaluate its security features compared to standard digital signature approaches.

ANSWER:

```
# ============================================
# ELGAMAL DIGITAL SIGNATURE IMPLEMENTATION
# ============================================

import hashlib
import random
import math


# ============================================
# HASH FUNCTION
# ============================================

def hash_message(message):
    return int(
        hashlib.sha256(message.encode()).hexdiges
        16
    )


# ============================================
# ELGAMAL KEY GENERATION
# ============================================

def generate_keys():
    # Prime number
    p = 467

    # Generator
    g = 2
```

```
Console                                                    Run

======================================================================
              ELGAMAL DIGITAL SIGNATURE
======================================================================

Key Generation
----------------------------------------------------------------------
Prime number (p) : 467
Generator (g)    : 2
Private Key (x)  : 361
Public Key (y)   : 203

Original Message:
This is a secure message.

Digital Signature:
r = 140
s = 325

======================================================================
VERIFYING ORIGINAL MESSAGE
======================================================================
```

```
# ============================================
# ELGAMAL SIGNATURE GENERATION
# ============================================

def sign_message(message, p, g, x):

    # Hash the message
    h = hash_message(message) % (p - 1)

    # Choose random k such that gcd(k, p-1) = 1
    while True:
        k = random.randint(2, p - 2)

        if math.gcd(k, p - 1) == 1:
            break

    # Calculate r
    r = pow(g, k, p)

    # Calculate modular inverse of k
    k_inverse = pow(k, -1, p - 1)

    # Calculate s
    s = ((h - x * r) * k_inverse) % (p - 1)

    return r, s


# ============================================
# SIGNATURE VERIFICATION
# ============================================
```

```
Console                                                    Run

ELGAMAL DIGITAL SIGNATURE:
- Uses a private key for signing.
- Uses a public key for verification.
- SHA-256 is used to hash the message.
- Provides message integrity and authentication.
- Random value k is required for every signature.
- Reusing k can reveal the private key.

COMPARISON WITH STANDARD DIGITAL SIGNATURES:
- ElGamal is based on the discrete logarithm problem.
- RSA signatures are based on integer factorization assumptions.
- Modern schemes such as RSA-PSS and ECDSA provide stronger
  standardized security than this simplified implementation.
- Elliptic Curve based signatures generally provide smaller keys
  with strong security.

======================================================================
                        CONCLUSION
======================================================================

ElGamal digital signatures provide authentication and integrity.
A modified message fails signature verification.

However, real-world systems should use standardized and tested
cryptographic libraries such as RSA-PSS, ECDSA, or Ed25519 rather
than implementing the algorithm manually.

Run completed in 1893ms
```